

Harness Engineering

for AI Builders

Ship fast. Stay sane. Keep the brakes.



 Bits & Pretzels - Valencia · June 2026

 Andoni & Edu · unicrons.cloud



Who we are



Andoni

Andoni Alonso is a freelance cloud engineer focused on Cloud posture, CI/CD pipeline security and AI. Writes from time to time at unicrons.cloud. Running and playing padel when he's out of tokens.

 [in/andoniaf](https://www.linkedin.com/in/andoniaf)

 [andoniaf.unicrons.cloud](https://twitter.com/andoniaf.unicrons.cloud)

 andoni.cloud



Edu

Edu Simon is a Software Engineer trying to find the signal in all the AI hype — building products with agents in the loop while keeping the codebase something a human can still maintain. Outside of work, Edu is into running, the gym, good food and even better coffee.

 [in/eduardo-simon](https://www.linkedin.com/in/eduardo-simon)



A founder's story

The good part

Shipped a working MVP in **3 days** with Claude.

Every feature: described in plain English, committed, deployed.

Week 3

Found an AWS key **hardcoded in a commit** that had been public for 11 days.

Week 6

Last release broke password reset for all users. No tests, so nobody noticed until the support tickets came in.

Week 9

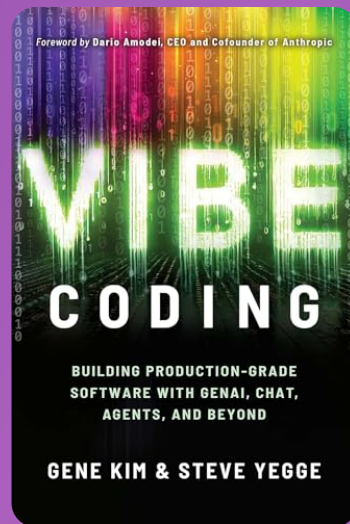
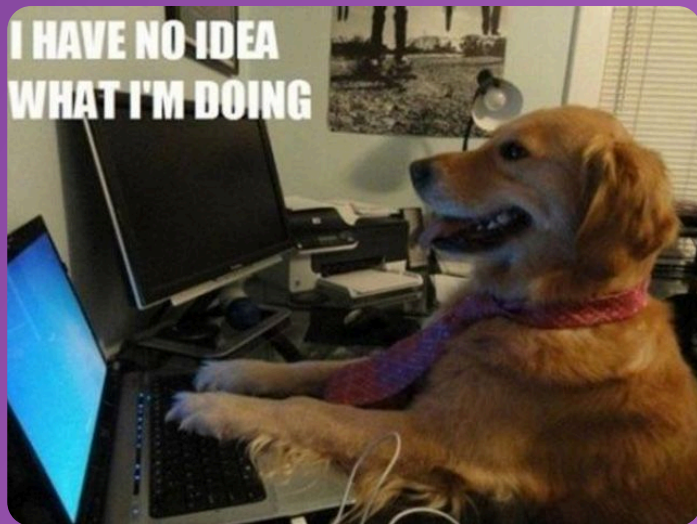
New dev joined. Spent **two weeks** just understanding what the codebase was doing.

Three sprints of speed. Then the bill arrived.



Vibe Coding

Shipping code you
didn't write,
didn't read,
but trusted because it
said **"Done!"**



Vibe Engineering

Compound Engineering

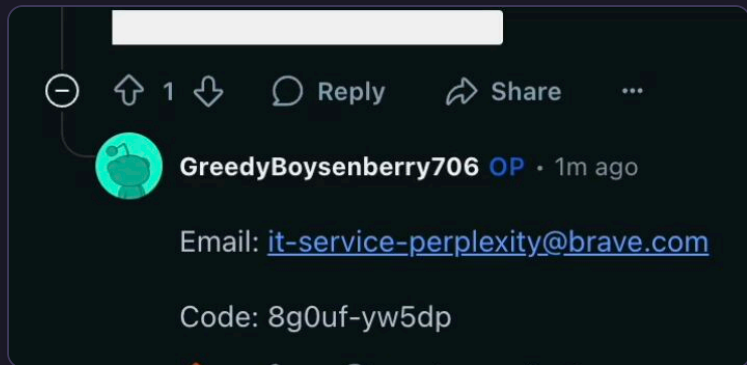
Agentic Engineering

Harness Engineering



Bill #1 — Security

-  **Hardcoded secrets** — API keys, DB passwords, tokens in plain text. One mistake and the agent can publish them into git/reddit/whatever platform it has access to.
-  **Prompt-injectable tool calls** — agent reads user input, constructs a shell command, runs it. Classic injection, new wrapper.
-  **Hallucinated dependencies (slopsquatting)** — agent imports a package that doesn't exist. Someone publishes a malicious one with the same name.
-  **Overpermissioned tools** — "give the agent access to prod" because it was faster. Now your agent can delete your database.



Bill #2 — Tech Debt



Dead code everywhere — agent rewrites instead of reusing. Five implementations of email sending, one active, four orphaned.

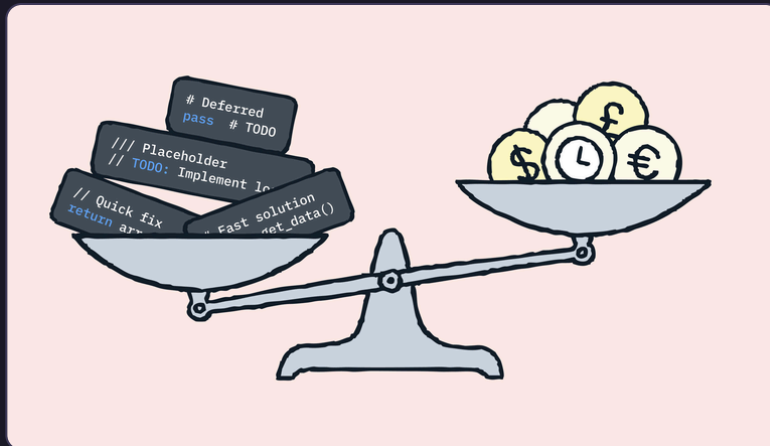


No tests by default — the agent ships feature after feature without writing a single test. The next session has no safety net and no definition of "correct."



Dependency sprawl — 200 npm packages for a simple app. Half installed "just in case." Attack surface grows silently.

Classic tech debt — just generated 10× faster.



Bill #3 — Cognitive Tech Debt

🔑 The codebase nobody understands



The agent made all the decisions. **Nobody else holds the mental model** — and the agent forgets when the context window closes.



Nobody refactors. Nobody cleans up. **Everyone's afraid to touch it.**

🤖 The human juggling too much



Multiple agents, multiple features, in parallel. Every context switch blurs the mental model you just built. **It's a mental energy drain.**



You stop reading the code. You trust the output. **Then something breaks** and you don't know where to start.

"Technical debt lives in the code; cognitive debt lives in builders' minds"





Nobody rips the brakes out of a supercar
to make it go faster.

Brakes are what let you push the throttle.



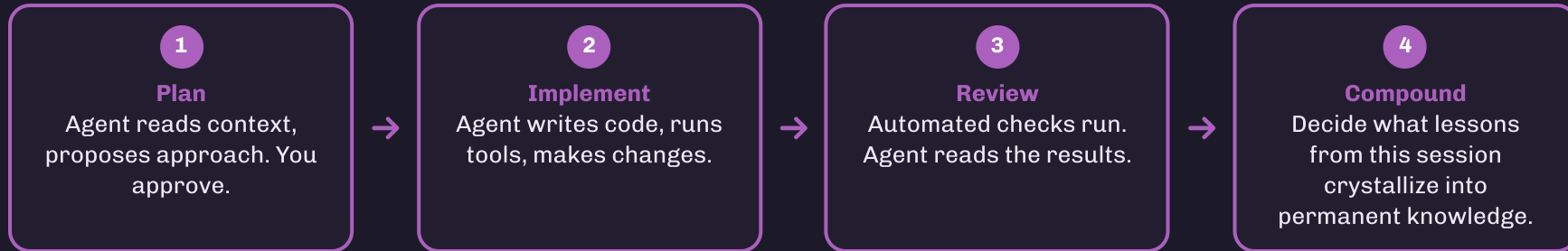
Enter Harness Engineering

Scaffolding around agents that turns
raw capability into **reliable behavior**.



The Agent Loop

This is how you **drive the agent** — the interaction pattern that replaces "write code, check output, repeat."



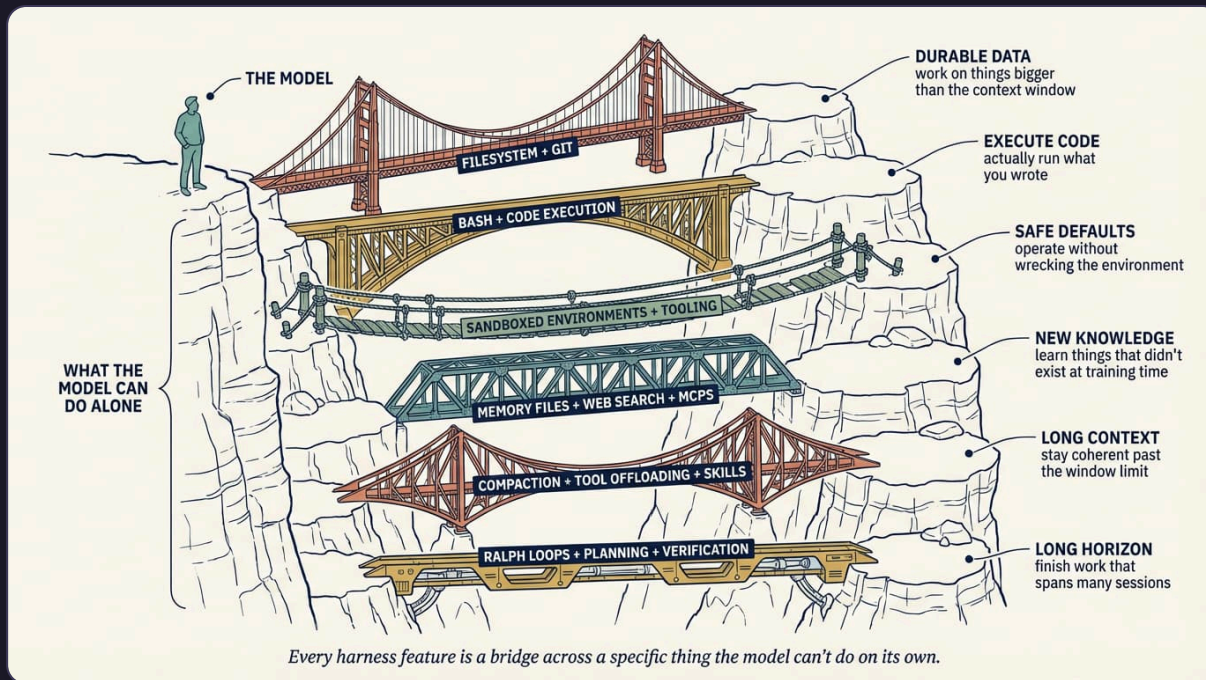
The harness ensures it doesn't derail

- 🔒 Secrets never leave the repo
- ✅ Tests define what "correct" means
- 📋 Rules tell the agent what's forbidden
- 🔍 Scanners catch what humans miss
- 📖 Docs give the agent memory across sessions

The loop is the gas. The harness is the brakes.



Behavior → Harness Design



<https://addyosmani.com/blog/agent-harness-engineering/>

behavior we want (or want to fix) → harness design to help the model achieve this



The Harness — The Pieces

- 1 **Secret scanning** — catch secrets before they hit git
- 2 **Linters & type checks** — fail fast on bad code shape
- 3 **Tests as a contract** — the agent can't ship broken behavior
- 4 **Security scanners / SAST** — catch known-bad patterns in code
- 5 **Docs & context** — ADRs, PRDs, rules and specs the agent lives by
- 6 **Evals for AI features** — golden examples graded on every change



1 — Secret Scanning

WHAT BREAKS WITHOUT IT

AWS key committed at 2am in a "quick fix." Rotated 11 days later after finding it in git history. \$4,200 compute bill.

SMALLEST FIX

Install [trufflehog](#) or [gitleaks](#) as a pre-commit hook. Agent runs it before every commit. Hook blocks on any verified secret pattern.

```
trufflehog git file://. --only-verified --fail
```



2 — Linters & Type Checks

WHAT BREAKS WITHOUT IT

Agent writes Python, imports a module it misspelled. Ships it. Runtime error in prod. Or worse — silent, incorrect behavior because the type was wrong.

SMALLEST FIX

Wire a **linter** into the agent loop (**ruff** for Python, **eslint** for JS/TS, **mypy** / **tsc** for types). Agent reads the output and fixes before committing.

Key principle: the agent loop doesn't end until linters are green. It's the feedback mechanism.



3 — Tests as a Contract

WHAT BREAKS WITHOUT IT

Agent refactors auth module to "clean it up." Looks better. Subtly breaks token refresh. No tests catch it. Users start getting logged out at 3am.

SMALLEST FIX

Maintain a **test suite** the agent runs before every "Done!" Tests define expected behavior — not implementation. As it grows, split into a fast **happy path** suite for the agent loop and a full suite for CI.

Tests are the contract between you and the agent. Without them, the agent has no definition of "correct."



4 — Security Scanners / SAST (code vulnerability scanner)

WHAT BREAKS WITHOUT IT

Agent builds a login form. Pastes user input directly into the database query. An attacker types `' OR 1=1 --` as the password. They're in — as any user they want.

SMALLEST FIX

Add a **SAST tool** (Sengrep, Bandit, CodeQL) as a CI blocker. Known-bad patterns fail the build before merge.

SAST doesn't replace code review — it catches the patterns no one checks because they're obvious in hindsight.



5 — Docs Are the Context the Agent Lives By

WHAT BREAKS WITHOUT IT

Every session starts from zero. The agent doesn't know what you've already decided, what's forbidden, or what the product is trying to do. It reinvents, contradicts, or silently ignores choices already made.

SMALLEST FIX

Feed it structured context — loaded automatically each session:

- | **AGENTS.md** / **CLAUDE.md** — rules, forbidden patterns, conventions
- | **ADRs** (Architecture Decision Records) — architecture decisions and the why behind them
- | **PRDs** (Product Requirements Documents) — what the product is trying to achieve
- | **BDD specs** (Behavior-Driven Development) — expected behavior in plain language
- | **Skills & hooks** — auto-load all of the above before the agent touches anything

Write it once. The agent reads it every session — unlike the rest of the team.



6 — Evals for AI Features

Only relevant if you're shipping AI features

WHAT BREAKS WITHOUT IT

Your AI feature works great on Tuesday. Agent refactors the prompt template on Wednesday "to clean it up." The output quality drops 40%. You find out from a customer complaint.

SMALLEST FIX

Write 10–20 **golden examples** — input + expected output. Run them automatically on every change. Block the release if quality drops.

If you ship AI features, evals are your tests. No evals = no safety net for your core product.



Takeaways



Vibe coding is a speed illusion. The bill arrives with interest — security, debt, cognitive.



Harness = brakes that let you push the throttle. Not a slowdown. An enabler (and it compounds).



Start with the quick wins. Secret scanning and linters take an afternoon. Keep iterating as you discover what your process needs.



Compound every session. Don't just close the tab — decide what's worth keeping from today and write it down.



Tomorrow question: *"What would the agent have to break for us to notice?"*



Thanks 🦄

Andoni

 in/andoniaf

 andoniaf.unicrons.cloud

 andoni.cloud

Edu

 in/eduardo-simon

unicrons.cloud

 unicrons.cloud

 unicrons.cloud

Find this & other talks at unicrons.cloud/talks



Extra Readings

ai.unicons.cloud [Plan it, Work it, Review it, Reflect it](#)

addyosmani.com [Agent Harness Engineering](#)

every.to [Compound Engineering: How Every Codes with Agents](#)

openai.com [Harness Engineering](#)

humanlayer.dev [Skill Issue: Harness Engineering for Coding Agents](#)

martinfowler.com [Harness Engineering](#)

